Chapter 18 - Spring MVC + Hibernate/JPA Integration

1. Overview

Spring MVC handles web requests while Hibernate/JPA manages persistence. A typical layered architecture separates Controller, Service and Repository (DAO) responsibilities.

2. Request Flow

```
Browser
   |
DispatcherServlet
   |
Controller
   |
Service (@Transactional)
   |
Repository / DAO
   |
EntityManager / Session
   |
Database
```

3. Layer Responsibilities

Controller: Receives HTTP requests and returns views or JSON. Service: Contains business logic and transaction boundaries. Repository/DAO: Executes persistence operations using EntityManager or Session.

4. Transaction Flow

```
Request
   |
Controller
   |
@Transactional Service
   |
Begin Transaction
   |
Repository
   |
Database
   |
Commit / Rollback
```

5. EntityManager / Session Lifecycle

Within a transaction, Spring provides an EntityManager (or Hibernate Session). Managed entities remain in the Persistence Context until the transaction completes or the context is cleared.

Layer	Common Annotation
Controller	@Controller / @RestController
Service	@Service
Repository	@Repository

Transaction	@Transactional
Dependency Injection	@Autowired / constructor injection

6. Typical Repository Example

```
@Repository
public class EmployeeRepository {

    @PersistenceContext
    private EntityManager em;

    public Employee find(Integer id){
        return em.find(Employee.class, id);
    }
}
```

7. Configuration

In classic Spring MVC applications, beans can be configured using XML or Java configuration. Modern Spring Boot applications typically rely on auto-configuration and application.properties/application.yml.

8. Best Practices

• Keep controllers thin. • Put business logic in services. • Use @Transactional at the service layer. • Prefer constructor injection. • Avoid exposing entities directly from REST APIs when DTOs are more appropriate.

9. Real-Time Scenario

A user submits an employee update form. The Controller validates input, the Service starts a transaction, the Repository updates the entity, Hibernate performs dirty checking, SQL is executed during flush/commit, and the transaction completes.

10. Interview Questions

1. Explain Spring MVC architecture. 2. Why use @Transactional on the Service layer? 3. Difference between @Repository and @Service? 4. Where should business logic reside? 5. How does Spring integrate with Hibernate/JPA?